Análise Comparativa de Arquiteturas de Redes Neurais para Classificação de Perfis de Jogadores

Pedro G. Gonçalves

Instituto de Geociências e Ciências Exatas – Universidade Estadual Paulista (UNESP)
Rio Claro – SP – Brazil

pedro.g.goncalves@unesp.br

Abstract. *One of the main challenges in the digital gaming industry is understanding player preferences in order to offer experiences that promote high levels of immersion and satisfaction. In this context, this work proposes an approach focused on the classification of player profiles. To this end, different artificial neural network models were implemented and compared, aiming to identify behavioral patterns and individual preferences in a dataset originating from the Overlord system. The obtained results enabled an analysis of the approach for hyperparameter validation, the understanding of the influence of preprocessing, the verification of which models achieved the best performance, and the comprehension of the dataset's influence even with variations in neural networks, in addition to identifying models that outperform others. Thus, this study contributes to the development of more adaptive and user-centered systems by providing insights for the selection of neural network architectures, hyperparameter calibration, and proper data processing — essential elements for improving the personalization of the experience in digital games.*

Resumo. *Um dos principais desafios da indústria de jogos digitais é compreender as preferências dos jogadores, a fim de oferecer experiências que promovam elevados níveis de imersão e satisfação. Nesse contexto, este trabalho propõe uma abordagem voltada à classificação de perfis de jogadores. Para isso, foram implementados e comparados diferentes modelos de redes neurais artificiais, com o objetivo de identificar padrões comportamentais e preferências individuais em um banco de dados oriundo do sistema Overlord. Os resultados obtidos permitiram uma análise de abordagem para validação de hiperparâmetros, para o entendimento da influência do pré-processamento, para verificar quais modelos alcançaram melhor desempenho, e para entender a influência do banco de dados mesmo com a variação de redes neurais, além da identificação de modelos que se sobressaem em relação aos demais, contribuindo, assim, para o desenvolvimento de sistemas mais adaptativos e centrados no usuário, ao fornecer subsídios para a escolha de arquiteturas de redes neurais, a calibração de hiperparâmetros e o tratamento adequado dos dados, elementos essenciais para melhorar personalização da experiência em jogos digitais.*

1. Introdução

A indústria de jogos digitais, que projeta expandir de US$ 25,5 bilhões para US$ 37,74 bilhões até 2035 [Global Growth Insights 2024], apresenta o desafio de atender

a públicos diversificados. Para isso, a adaptação de conteúdo é uma estratégia fundamental para aumentar a imersão e a satisfação do jogador, modificando elementos da experiência conforme seu perfil, comportamento e desempenho [Martins 2021, Balda 2020, de Lima et al. 2022].

Assim, diversas técnicas são utilizadas, desde heurísticas e lógica *fuzzy* até algoritmos de Aprendizado de Máquina (AM) e Aprendizado Profundo (AP), sendo estas últimas frequentemente mais eficazes para modelar a complexa interação do jogador [Rosa et al. 2023, Chrysafiadi et al. 2023, Chanel and Lopes 2020, de Lima et al. 2022]. No entanto, a eficácia dessas abordagens em cenários de alta complexidade e variabilidade, como os proporcionados por sistemas de Geração Procedural de Conteúdo (PCG), que atuam como orquestradores, ainda permanece pouco explorada, apesar de seu elevado interesse comercial e científico [Shaker et al. 2016, de Lima et al. 2022, Liapis et al. 2019].

Este projeto propõe investigar e comparar abordagens para que possa classificar perfis de jogadores, possibilitando em trabalhos futuros adaptação dinâmica de conteúdo integradas a um orquestrador de PCG, o *Overlord* [Pereira et al. 2024]. O sistema é responsável por coordenar dinamicamente múltiplas facetas de conteúdo gerado proceduralmente — Níveis (masmorras com quebra-cabeças de portas trancadas e posicionamento de inimigos), Narrativa (missões) e Regras (inimigos com diferentes atributos e mecânicas) — garantindo coerência estrutural entre os geradores e adequação ao perfil do jogador.

O sistema realiza a classificação do jogador com base em métricas da partida e, a partir dessa predição, orquestra de forma integrada os geradores de níveis, narrativa e regras. O objetivo central é adaptar múltiplos elementos do jogo de maneira consistente e personalizada, considerando distintos perfis e estilos de jogabilidade.

Para a condução dos experimentos, foi utilizada uma base de dados composta por aproximadamente 120 instâncias de gameplay (registros de sessões de jogo), posteriormente expandida para mais de 1000 instâncias por meio de técnicas estatísticas de aumento de dados (data augmentation), conforme metodologia validada na literatura [Pereira et al. 2025]. Esse procedimento visa mitigar os efeitos do tamanho reduzido da amostra original, permitindo uma avaliação mais robusta dos modelos de AM.

Para isso, arquiteturas de redes neurais artificiais representativas de modelos feedforward (FNN), recorrentes (RNN, LSTM, GRU), convolucionais (CNN) e temporais (TCN) foram adaptadas e avaliadas [Romero-Mendez et al. 2023, Rymarczyk et al. 2021, Heinz et al. 2017, Azevedo et al. 2020, You et al. 2019]. Também foram investigadas arquiteturas híbridas, como FNN-LSTM, FNN-GRU, CNN-LSTM, CNN-GRU, RNN-LSTM, RNN-GRU, TCN-LSTM e TCN-GRU [Chanel and Lopes 2020, An et al. 2019, Zhou et al. 2025, Kodiyan et al. 2017].

A avaliação experimental foi conduzida por meio de validação cruzada (*k-fold cross-validation*) e divisão treino-teste, empregando métricas de desempenho como acurácia, precisão, *recall*, *F1-score*, *gap* e coeficiente Kappa. Essa abordagem permite examinar tanto a capacidade preditiva quanto a eficiência dos modelos. Também foi utilizada a matriz de confusão para análise detalhada dos padrões de classificação.

Em etapa posterior, modelos com melhores desempenhos foram selecionados para

que possam ser testados em um cenário mais adequado e, futuramente, integrados ao sistema de adaptação do orquestrador, possibilitando sua avaliação em um cenário aplicado. Desse modo, será possível analisar impactos na qualidade da classificação de perfis e na adequação da adaptação do conteúdo ao jogador.

As principais contribuições deste trabalho são: (i) fornecer, por meio dos resultados obtidos, uma análise fundamentada dos pontos fortes e das limitações de cada abordagem em um cenário de orquestração procedural, contribuindo para a área de jogos sérios e entretenimento [Schrader 2022]; (ii) gerar subsídios teóricos e práticos para o desenvolvimento de sistemas adaptativos mais robustos; (iii) oferecer uma contribuição relevante para a área de AM, ao avaliar a capacidade de generalização e a eficiência de diferentes arquiteturas em ambientes interativos e estocásticos; e (iv) obter uma análise do procedimento de determinação dos hiperparâmetros, suas influências e a do conjunto de dados nos modelos.

O artigo está organizado da seguinte forma: a Seção 2 descreve a metodologia; a Seção 3 discute os experimentos e resultados; e a Seção 4 apresenta as conclusões.

2. Metodologia

Esta seção aborda os trabalhos relacionados e os métodos adotados na etapa de desenvolvimento e treinamento das redes neurais.

2.1. Tecnologia

As redes neurais foram desenvolvidas utilizando a linguagem de programação *Python*, no ambiente de desenvolvimento *Google Colab*[1].

2.2. Base de dados

O *Overlord* é um orquestrador de conteúdo responsável por coordenar três facetas criativas do jogo: (i) *níveis*, que produz masmorras compostas por salas contendo inimigos, NPCs e itens; (ii) *regras*, responsável por selecionar os parâmetros dos inimigos, tais como saúde, velocidade de movimento e dano causado; e (iii) *narrativa*, que define o *template* da missão de acordo com o perfil do jogador [Pereira et al. 2022].

O sistema considera o perfil do jogador para pós-processar o conteúdo gerado, caracterizando-se como um sistema adaptativo. Sua eficácia foi avaliada em estudos anteriores, com resultados positivos em termos de diversão e adaptação ao perfil do jogador [Pereira et al. 2022, Teoi et al. 2024].

Com base nesse sistema, foi utilizado neste trabalho um conjunto de dados coletado a partir de sessões de jogo orquestradas pelo *Overlord*, nas quais foram registradas e analisadas as experiências dos jogadores nos estudos supracitados, resultando em um total de 119 instâncias.

2.3. Arquiteturas utilizadas

Neste trabalho, foram selecionadas diferentes arquiteturas com o objetivo de identificar quais são mais adequadas ao contexto do problema investigado. As escolhas foram baseadas em modelos previamente avaliados na literatura em cenários

[1]Google Colab. Disponível em: https://colab.research.google.com/. Acesso em: 6 maio 2026.

semelhantes, bem como em combinações híbridas propostas para analisar seu comportamento em conjunto [Romero-Mendez et al. 2023, Rymarczyk et al. 2021, Heinz et al. 2017, Azevedo et al. 2020, You et al. 2019, Chanel and Lopes 2020, An et al. 2019, Zhou et al. 2025, Kodiyan et al. 2017].

Entre as arquiteturas consideradas, destacam-se: (i) redes neurais feedforward; (ii) redes neurais recorrentes e suas variantes; (iii) redes neurais convolucionais; e (iv) redes temporais convolucionais. Além dessas, também foram avaliados modelos híbridos que combinam diferentes arquiteturas, tais como CNN-LSTM, CNN-GRU, RNN-LSTM, RNN-GRU, FNN-LSTM, FNN-GRU, TCN-LSTM e TCN-GRU. A seguir é detalhado cada uma dessas arquiteturas e suas respectivas motivações para adoção neste trabalho.

2.3.1. LSTM

Recurrent Neural Network (RNN) é especializada em dados sequenciais, mantendo uma memória interna do contexto anterior. A variante *Long Short-Term Memory* (LSTM) foi desenvolvida para superar limitações das RNNs padrão, como o desaparecimento do gradiente, sendo capaz de aprender dependências de longo e curto prazo em sequências temporais de forma mais estável [Hochreiter and Schmidhuber 1997].

Na literatura, uma de suas aplicações foi utilizada no contexto de entender as preferências de clientes em compras online, aprendendo correlações temporais no histórico de compras de cada usuário [Heinz et al. 2017].

2.3.2. GRU

Gated Recurrent Unit (GRU) é uma arquitetura de rede neural recorrente que simplifica o modelo LSTM, utilizando dois mecanismos de controle — portas de reset e update — para capturar dependências temporais em sequências, com menor custo computacional [Rymarczyk et al. 2021].

Assim, GRU é destacado como uma versão simplificada do LSTM, oferecendo desempenho similar com menor custo computacional, sendo aplicado a tarefas como previsão de vendas e segmentação temporal para perfilamento[Rymarczyk et al. 2021].

2.3.3. FNN

Feedforward Neural Network (FNN) é uma arquitetura na qual as conexões entre os nós não formam ciclos, permitindo que os dados fluam da camada de entrada para a camada de saída através de camadas ocultas, o que a torna particularmente eficaz para tarefas de classificação de padrões e regressão [Romero-Mendez et al. 2023].

Desse modo, na literatura a FNN foi aplicada para classificar jogadores em níveis de habilidade com base em métricas extraídas durante a partida, demonstrando alta precisão na tarefa de classificação supervisionada [Romero-Mendez et al. 2023].

2.3.4. CNN

Convolutional Neural Network (CNNs) é projetada para processar dados com estrutura de grade, como imagens ou séries temporais estruturadas, utilizando camadas convolucionais e de agrupamento (pooling) para extrair características hierárquicas e localmente invariantes de forma eficiente [Miyazaki 2017].

Por exemplo, as CNNs aplicadas à identificação de perfis de clientes por meio da classificação de imagens postadas em redes sociais, em categorias pré-definidas de interesse [Azevedo et al. 2020].

2.3.5. RNN

Dados de consumo e interação de clientes possuem natureza sequencial, exigindo modelos capazes de capturar dependências temporais. Redes neurais recorrentes (RNNs) foram concebidas para essa finalidade, processando sequências por meio de passos temporais encadeados que integram informações atuais e históricas [Zhou et al. 2025].

No contexto de farmácias de varejo, aplicou-se LSTM (uma variante da RNN) para modelar o histórico de compras de medicamentos, que apresenta padrões periódicos influenciados por ciclos de tratamentos crônicos [Zhou et al. 2025].

2.3.6. TCN

As Temporal Convolutional Networks (TCNs) emergem como uma alternativa eficiente às redes recorrentes para modelagem de sequências, combinando convoluções causais e dilatadas para capturar dependências temporais de longo prazo com receptivo exponencial, evitando o desaparecimento do gradiente [You et al. 2019].

Foi proposto a arquitetura TCN para interesses de curto prazo (dentro de uma sessão), alcançando maior velocidade e menor consumo de memória [You et al. 2019].

2.3.7. Híbridas

Arquiteturas híbridas combinam diferentes abordagens para potencializar a modelagem de dados sequenciais, superando limitações de modelos individuais. Uma estratégia comum é a integração de CNNs com RNNs, na qual as CNNs extraem padrões contextuais locais enquanto as camadas recorrentes capturam dependências temporais de longo prazo [Chanel and Lopes 2020].

No contexto de classificação de perfis de autores em redes sociais, o uso de redes bidirecionais com GRU e mecanismos de atenção demonstrou eficácia, pois a bidirecionalidade captura o contexto completo da sequência textual enquanto a atenção pondera os elementos mais relevantes para a classificação [Kodiyan et al. 2017]. De forma complementar, a LSTM foi empregada em sistema de perfilamento de clientes para farmácias, modelando padrões periódicos de compra [Zhou et al. 2025].

Em suma, as arquiteturas apresentadas modelam padrões sequenciais, temporais

ou hierárquicos em dados, sendo que todas foram utilizadas na literatura no contexto de identificação de padrões de usuários. No contexto deste trabalho — identificação de perfis de jogadores — essa capacidade é central, pois o comportamento do jogador evolui em sequências de ações (partidas) e ao longo de múltiplas sessões. Assim, LSTM e GRU capturam dependências temporais com eficiência; RNN fornece a base conceitual sequencial; TCN atua em interesses de curto prazo com alta velocidade; FNN realiza classificação supervisionada; CNN identifica perfis de interesse; e arquiteturas híbridas combinam padrões locais com modelagem temporal de longo prazo.

2.4. Abordagem

Para o desenvolvimento do modelo responsável por aprender a realizar a adaptação de conteúdo por meio do perfilamento, foi necessário compreender e definir os perfis dos jogadores. Nesse sentido, adotou-se a definição dos quatro fatores de motivação proposta por Yee [Yee and Ducheneaut 2018]: ***A**chievement*: refere-se a jogadores que priorizam conquistas e o fortalecimento de seus personagens; ***C**reativity* caracteriza jogadores que valorizam personalização e exploração; ***I**mmersion* descreve jogadores interessados na narrativa e na incorporação de seus personagens; e ***M**astery* representa jogadores motivados por desafios, resolução de problemas e experiências estratégicas.

Cada fator recebe um valor único entre 1 e 4, que representa sua influência, sendo 1 o mínimo e 4 o máximo. Por exemplo, um jogador do tipo *Mastery* receberia valor 4 para esse fator, enquanto os demais variariam entre 1 e 3. Essa é a representação utilizada pelo *Overlord* e, neste trabalho, ela foi mantida para facilitar o entendimento e a comparação dos resultados [Pereira et al. 2022].

A partir dessas definições, o problema foi formulado como uma tarefa de classificação multiclasse, na qual o objetivo é identificar o perfil predominante do jogador com base em suas métricas da partida. Para isso, foi estruturado um pipeline composto por: (i) pré-processamento dos dados, incluindo a remoção de variáveis irrelevantes, redundantes ou não informativas; (ii) desenvolvimento de diferentes modelos de redes neurais, conforme descrito na Seção 2.2; (iii) ajuste de hiperparâmetros, conforme descrito na Seção 3.2; e (iv) avaliação de desempenho e seleção do melhor modelo. O fluxo do processo proposto é apresentado na Figura 1.

3. Experimentos e Resultados

Esta seção descreve como os dados foram utilizados, incluindo os procedimentos de pré-processamento adotados, bem como o desenvolvimento e a avaliação dos modelos e de seus respectivos experimentos. Além disso, são apresentados os resultados obtidos.

3.1. Configuração experimental

Os experimentos foram conduzidos utilizando o conjunto de dados composto por métricas de gameplay e durante a etapa de pré-processamento, foram removidas variáveis irrelevantes, redundantes ou não informativas, incluindo atributos de metadados e variáveis não utilizadas na modelagem.

Para aumentar a capacidade de generalização dos modelos, foi aplicada a técnica de aumento de dados exclusivamente ao conjunto de treinamento, evitando vazamento de informação e preservando a validade da avaliação experimental [Ricci et al. 2010].

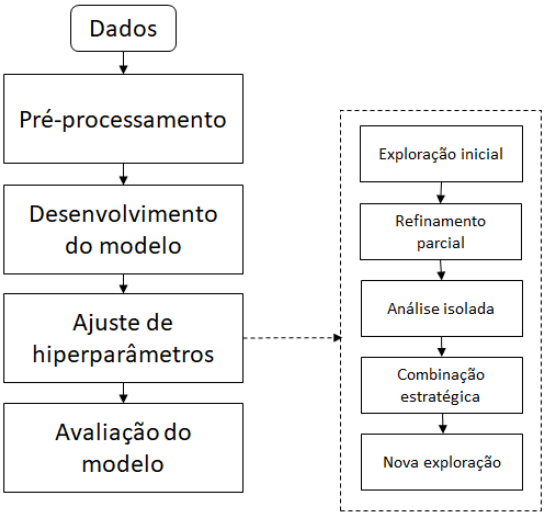


Figura 1. Fluxo do desenvolvimento dos modelos de redes neurais, com destaque para a estratégia de ajuste de hiperparâmetros.

O aumento foi realizado por meio da adição de ruído às instâncias originais e da aplicação de transformações de escalonamento com fatores de 0,9, 0,95, 1,05 e 1,1 [Pereira et al. 2025].

Como resultado, o conjunto de treinamento foi expandido para 1309 instâncias, representando um aumento significativo em relação ao conjunto original [Pereira et al. 2025].

Durante os experimentos, foi adotada uma semente fixa (*random seed*) igual a 42, visando garantir a reprodutibilidade dos resultados. Os modelos foram treinados por até 500 épocas (*epochs*), permitindo a convergência do processo de otimização. Nas arquiteturas que exigiam definição explícita de ativação nas camadas ocultas, foi utilizada a função ReLU (*Rectified Linear Unit*). Já arquiteturas recorrentes, como LSTM e GRU, utilizaram suas funções de ativação internas padrão.

As arquiteturas híbridas foram implementadas por meio de empilhamento sequencial de módulos neurais, no qual a saída intermediária de uma rede é utilizada como entrada para a subsequente. O treinamento foi realizado de maneira conjunta (*end-to-end*), sem utilização explícita de concatenação de representações.

A avaliação dos modelos foi conduzida por meio de validação cruzada (*5-fold cross-validation*) e divisão treino-teste, utilizando 70% das instâncias para treinamento e 30% para teste. Foram utilizadas métricas de desempenho incluindo acurácia, precisão, *recall*, *F1-score*, *gap* e coeficiente Kappa. O *gap* foi calculado como a diferença entre a acurácia média de treinamento e a acurácia média de validação. A métrica *CV Mean* representa a média da acurácia obtida durante a validação cruzada, enquanto *CV Std* representa o desvio padrão entre as folds, sendo utilizada como indicador de estabilidade do modelo.

Para uma análise mais detalhada do desempenho, também foram utilizadas matrizes de confusão, permitindo a identificação de padrões de classificação e possíveis erros sistemáticos dos modelos.

Assim, na análise, seguindo a orientação da literatura [Kline 2016], a validação de modelos não deve apoiar-se unicamente em métricas globais de desempenho, mas também considerar a coerência e a interpretabilidade dos resultados no contexto aplicado. Para subsidiar a análise dos resultados, estabeleceram-se os seguintes critérios de interpretação das métricas de desempenho [Goodfellow et al. 2016, Bishop 2006, Kohavi 1995]:

- **Acurácia, F1-score, Kappa**: valores baixos indicam problema no modelo.
- **Precisão, Recall, CV Mean**: valores altos indicam bom desempenho.
- **Gap**: deve ser interpretado em conjunto com a acurácia de teste para avaliar o comportamento do modelo durante o treinamento, permitindo identificar se houve sobreajuste ou subajuste.
- **CV Std**: valores baixos indicam boa estabilidade do modelo.

3.2. Avaliação dos hiperparâmetros

Para a validação das redes, foi realizada a seleção do melhor modelo local por meio de busca de hiperparâmetros. Para isso, foi adotada uma estratégia híbrida que combina etapas sugeridas pela literatura [Bishop 2006, Hastie et al. 2009, Bergstra and Bengio 2012, Goodfellow et al. 2016] com um processo estruturado de exploração progressiva do espaço de hiperparâmetros.

O procedimento foi dividido nas seguintes etapas:

- (i) **Exploração inicial**: busca ampla no espaço de hiperparâmetros, baseada em recomendações da literatura [Goodfellow et al. 2016, Bishop 2006, Prince 2023], com o objetivo de identificar regiões promissoras de desempenho;
- (ii) **Refinamento parcial**: redução do espaço de busca com base nos melhores resultados observados na etapa anterior;
- (iii) **Análise isolada de hiperparâmetro**: variação controlada de um único hiperparâmetro por vez, mantendo os demais fixos, para avaliar sua influência individual;
- (iv) **Combinação estratégica (coarse-to-fine)**: ajuste conjunto dos principais hiperparâmetros selecionados, refinando combinações promissoras;
- (v) **Nova tentativa exploratória**: reexploração do espaço próximo aos melhores resultados para evitar ótimos locais e melhorar a generalização.

Ao final do processo, foi selecionado, para cada rede, o modelo que apresentou o melhor desempenho nas métricas de validação. O fluxo do procedimento proposto é apresentado na Figura 1.

3.3. Resultados

Nesta seção, são apresentados e analisados os resultados obtidos pelos diferentes modelos avaliados. Os desempenhos são consolidados na Tabela 1, a qual permite uma visão agregada dos intervalos correspondentes às melhores configurações de hiperparâmetros obtidas em cada etapa de otimização.

Tabela 1. Desempenho das arquiteturas avaliadas. Para cada modelo, são apresentados o intervalo valores mínimos e máximos das principais métricas (acurácia, F1-score, Kappa) e os indicadores de estabilidade da validação cruzada (CV Mean, CV Std).

Modelo	Acurácia	F1-score	Kappa	CV Mean	CV Std
LSTM	0,1567–0,2854	0,1007–0,2107	0,0654–0,0972	0,2246–0,9541	0,0108–0,0345
GRU	0,0933–0,2948	0,0608–0,2466	0,0174–0,0798	0,4331–0,8560	0,0124–0,0429
FNN	0,1455–0,2668	0,1232–0,2317	0,0483–0,0867	0,2543–0,4073	0,0242–0,2155
FNN+LSTM	0,0410–0,2593	0,0554–0,1954	0,0171–0,1122	0,2190–0,8179	0,0114–0,0563
FNN+GRU	0,0040–0,2369	0,0794–0,1638	0,0000–0,0955	0,2230–0,6443	0,0094–0,0728
CNN	0,1287–0,2780	0,0726–0,1778	–0,0097–0,0819	0,6095–0,8218	0,0176–0,1124
CNN+LSTM	0,1511–0,2817	0,1141–0,2025	0,0106–0,0733	0,6162–0,8387	0,0579–0,1061
CNN+GRU	0,1772–0,2817	0,1053–0,1762	0,0065–0,1035	0,4230–0,8756	0,0124–0,0665
RNN	0,0802–0,2743	0,0853–0,2294	0,0224–0,1278	0,4723–0,6095	0,0175–0,0451
RNN+LSTM	0,1289–0,2836	0,1056–0,2160	0,0182–0,1039	0,4095–0,5978	0,0184–0,0405
RNN+GRU	0,0970–0,2463	0,1001–0,2165	–0,0073–0,0817	0,4246–0,7535	0,0091–0,0660
TCN	0,0933–0,2425	0,0962–0,1310	–0,0176–0,0123	0,2129–0,6420	0,0121–0,0916
TCN+LSTM	0,0160–0,2687	0,0781–0,1923	–0,0284–0,0711	0,3821–0,6835	0,0179–0,0975
TCN+GRU	0,0125–0,2407	0,0806–0,7214	–0,0017–0,0619	0,2577–0,8667	0,0112–0,1394

3.3.1. Hiperparâmetros

Seguindo a estratégia de validação de hiperparâmetros, conduzida em cinco etapas progressivas, ao longo do processo observou-se a influência dos hiperparâmetros e o comportamento do conjunto de dados.

Primeiramente, ao analisar os otimizadores, percebeu-se que o SGD obteve os melhores resultados, sendo o primeiro entre as fases de forma predominante, seguido pelo Adam, que alcançou o segundo lugar com maior frequência e, em alguns casos, também ocupou a primeira posição. A maioria dos modelos apresentou o SGD como a melhor combinação; no entanto, em algumas situações, o Adam se destacou, embora com menor frequência. Outra configuração, com o RMSProp, apresentou alguns momentos em primeiro lugar, porém não foi a melhor combinação em nenhuma das análises gerais.

Como pode ser observado na Figura 2, apresenta-se a distribuição percentual dos otimizadores e sua predominância. A fase 5 não é apresentada na figura, pois nela foram testados novamente a taxa de aprendizado, o uso de dropout e o número de camadas, considerando como base o melhor resultado obtido até então.

Já na Tabela 2, identifica-se qual otimizador esteve mais frequentemente entre os melhores para cada modelo e qual resultou na melhor combinação geral. Em apenas dois casos, o Adam apresentou o melhor resultado — TCN e CNN+LSTM —, o que pode indicar que essas arquiteturas se beneficiam de maior estabilidade e adaptação no processo de otimização, características associadas a esse otimizador, nesse contexto. Por outro lado, o SGD com momentum foi predominante na maioria dos modelos, sugerindo que sua capacidade de manter a direção das atualizações.

Quanto ao número de neurônios, verificou-se uma variação entre 25, 30 e 50 nas arquiteturas clássicas, enquanto em alguns modelos híbridos os valores foram menores, variando entre 5, 8 e 16. Essa diferença pode ser explicada pelo fato de que valores mais elevados resultaram em pior desempenho, além de aumentarem o tempo de treinamento,

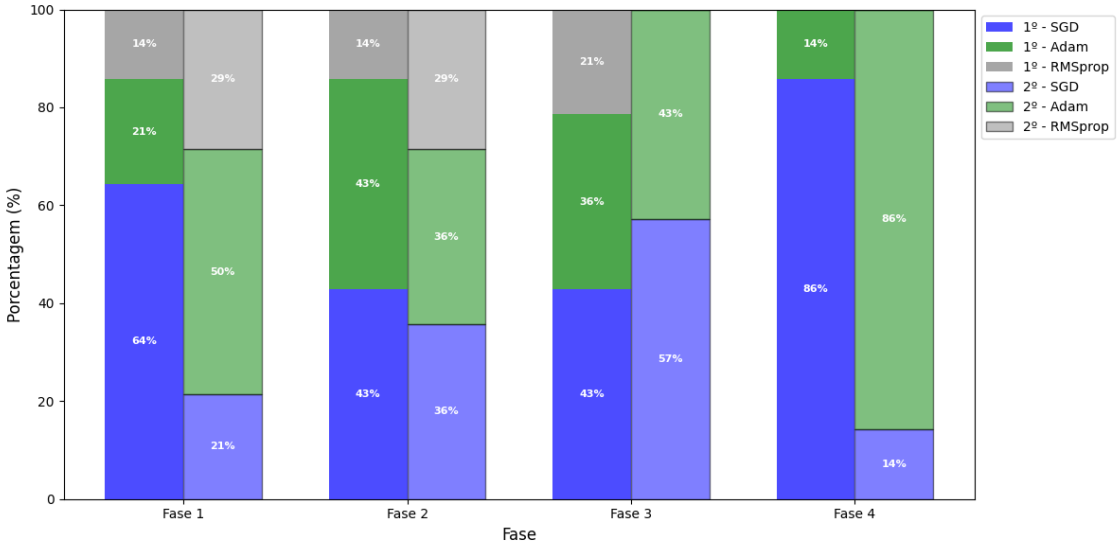


Figura 2. Distribuição percentual do desempenho dos otimizadores ao longo das etapas de validação de hiperparâmetros, para os dois melhores resultados de cada fase

sugerindo que modelos mais compactos são mais adequados ao problema.

Na Figura 3, apresenta-se a distribuição percentual dos melhores resultados para os hiperparâmetros taxa de aprendizado, número de camadas e regularização.

Em relação à taxa de aprendizado, observou-se que, na maioria dos casos, valores mais elevados resultaram em melhor desempenho. No entanto, taxas de aprendizado mais baixas também apresentaram bons resultados em cenários específicos, indicando que esse hiperparâmetro possui influência significativa e dependente da configuração do modelo.

No que diz respeito ao número de camadas, os resultados mostraram-se relativamente equilibrados. Ainda assim, observa-se que o uso de apenas uma camada é suficiente na maioria dos casos, o que pode estar relacionado ao tamanho do conjunto de dados, não justificando arquiteturas mais profundas.

Por fim, em relação à regularização, os melhores resultados foram obtidos sem a utilização de dropout ou com valores baixos. Esse comportamento indica que o modelo pode estar precisando de aprendizado mais estável e direto.

Na Tabela 3, estão explicitadas as melhores configurações de hiperparâmetros obtidas para cada arquitetura avaliada. Observa-se que a fase 5 concentrou o maior número de melhores configurações, totalizando cinco modelos, evidenciando que essa etapa teve maior impacto no desempenho final. Além disso, a partir da fase 3 concentra-se aproximadamente 71% do total de modelos (10 de 14), o que indica que a abordagem adotada a partir dessa etapa foi eficiente na identificação de configurações mais adequadas.

Em resumo, o SGD com momentum mostrou-se mais estável e apresentou melhores resultados médios nesse contexto. Esse comportamento pode ser explicado pelas características dos modelos utilizados, nos quais também se observou a necessidade de poucas camadas e baixa aplicação de regularização. Além disso, taxas de aprendizado mais baixas representam um processo de otimização mais gradual, no qual o modelo

Tabela 2. Melhor otimizador por modelo (média e pico)

Modelo	Melhor média	Melhor pico
LSTM	SGD	SGD
GRU	SGD	SGD
FNN	SGD	SGD
FNN+LSTM	SGD	SGD
FNN+GRU	Adam	SGD
CNN	Adam	SGD
CNN+LSTM	Adam	Adam
CNN+GRU	SGD	SGD
RNN	SGD	SGD
RNN+LSTM	SGD	SGD
RNN+GRU	SGD	SGD
TCN	Adam	Adam
TCN+LSTM	SGD	SGD
TCN+GRU	SGD	SGD

Tabela 3. Melhores configurações de hiperparâmetros por arquitetura

Modelo	Otimizador	LR	Hidden Size	Camadas	Dropout	Momentum	Fase
LSTM	SGD	0.0001	30	1	0.0	0.9	5
GRU	SGD	0.001	16	1	0.0	0.9	2
FNN	SGD	0.001	30	1	0.0	0.95	4
FNN+LSTM	SGD	0.001	LSTM: 50 / MLP: 25	2	0.0	0.9	4
FNN+GRU	SGD	0.0005	8	1	0.2	0.9	5
CNN	SGD	0.001	8	1	0.0	0.95	4
CNN+LSTM	Adam	0.0001	LSTM: 30 / CNN: 8	1	0.0	–	3
CNN+GRU	SGD	0.001	GRU: 25 / CNN: 16, 8	GRU:1 / CNN: 2	0.2	0.9	5
RNN	SGD	0.001	8	1	0.0	0.9	5
RNN+LSTM	SGD	0.001	8	1	0.0	0.9	4
RNN+GRU	SGD	0.001	8	1	0.0	0.9	2
TCN	Adam	0.0001	8	1	0.0	–	1
TCN+LSTM	SGD	0.001	16	1	0.3	0.9	2
TCN+GRU	SGD	0.0001	8	1	0.0	0.9	5

aprende de forma mais lenta, realizando atualizações menores nos gradientes e permitindo um ajuste mais refinado dos parâmetros.

3.3.2. Métricas

Agora, ao analisar as métricas, observa-se na Tabela 1 que, apesar de haver variação nos intervalos de desempenho entre as arquiteturas avaliadas, essas diferenças são sutis e, em muitos casos, sobrepostas. Métricas como acurácia, *F1-score* e Kappa apresentam faixas que, embora variem entre os modelos, permanecem próximas em magnitude, indicando um comportamento semelhante. Esse padrão sugere que os modelos tendem a convergir para níveis de desempenho comparáveis no contexto avaliado. Além disso, a variabilidade observada nos indicadores de validação cruzada reforça que não há ganhos expressivos e consistentes de uma arquitetura em relação às demais.

Porém, ao analisar atentamente as diferentes variações de cada modelo, foi possível identificar os melhores desempenhos. Nesse sentido, a Tabela 4 apresenta a

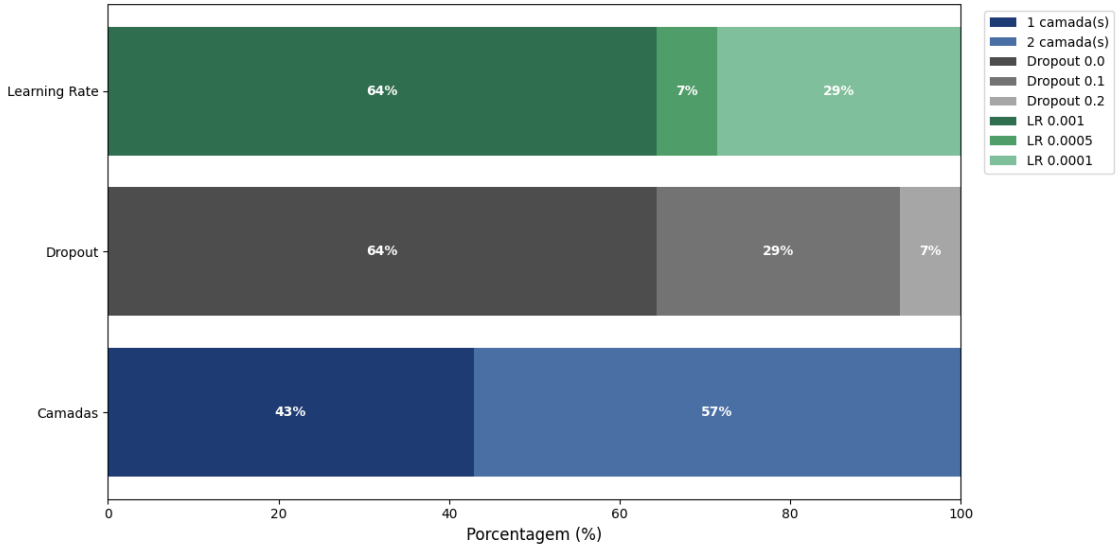


Figura 3. Distribuição percentual do desempenho das combinações de hiperparâmetros na fase 3 e fase 5, considerando taxa de aprendizagem, número de camadas e uso de dropout, para os dois melhores resultados de cada fase.

configuração final selecionada para cada arquitetura, considerando o melhor desempenho obtido na validação.

Tabela 4. Melhores resultados dos modelos (métricas de teste)

Modelo	Acc (Test)	F1	Prec (Test)	Recall (Test)	Kappa (Test)	Gap	CV Mean	CV Std
LSTM	0.2854	0.2097	0.3646	0.2854	0.0946	-0.0316	0.2246	0.0189
GRU	0.2948	0.2356	0.2466	0.2948	0.1200	0.2624	0.4538	0.0169
FNN	0.2668	0.1928	0.2035	0.2668	0.0812	0.1936	0.4073	0.0749
FNN+LSTM	0.2593	0.1825	0.1849	0.2593	0.0355	-0.0504	0.2190	0.0114
FNN+GRU	0.2369	0.1416	0.2090	0.2369	0.397	0.1169	0.3557	0.0210
CNN	0.2780	0.1778	0.3983	0.2780	0.0819	0.5243	0.7569	0.0971
CNN+LSTM	0.2817	0.2025	0.2958	0.2817	0.0544	0.3052	0.6263	0.0407
CNN+GRU	0.2817	0.1762	0.3142	0.2817	0.0817	0.1482	0.4286	0.0342
RNN	0.2743	0.2294	0.4968	0.2743	0.1278	0.4321	0.6213	0.0539
RNN+LSTM	0.2836	0.1772	0.1972	0.2836	0.1039	0.1872	0.4835	0.0316
RNN+GRU	0.2463	0.1798	0.1628	0.2463	0.0817	0.2168	0.4555	0.0427
TCN	0.2425	0.1033	0.0881	0.2425	0.0076	-0.0120	0.2129	0.0151
TCN+LSTM	0.2687	0.1923	0.2441	0.2687	0.0711	0.1421	0.3905	0.0283
TCN+GRU	0.2407	0.1783	0.174	0.2407	0.0349	0.0564	0.2818	0.0186

Ou seja, esses resultados indicam que a influência do conjunto de dados é significativa, visto que os modelos não apresentam diferenças expressivas de desempenho entre si. Isso sugere que limitações relacionadas aos dados podem estar impactando diretamente os valores das métricas, de modo que melhorias mais substanciais dependeriam, sobretudo, da resolução dessas questões, como qualidade, representatividade e balanceamento do conjunto de dados. Corroborando essa interpretação, as Matrizes de Confusão obtidas para todos os modelos apresentaram comportamento semelhante, conforme exemplificado na Figura 4, revelando que os erros de classificação se concentram nas mesmas classes independentemente do algoritmo utilizado, o que sugere que parte da dificuldade pode estar associada às características do conjunto de dados. Além disso, observou-se

baixa diversidade nas ordenações obtidas durante os experimentos, visto que, embora existam (4! = 24) permutações possíveis para a ordenação de quatro elementos, apenas oito combinações distintas foram produzidas, indicando que determinadas configurações ocorreram com maior frequência, enquanto outras não apareceram nas execuções realizadas.

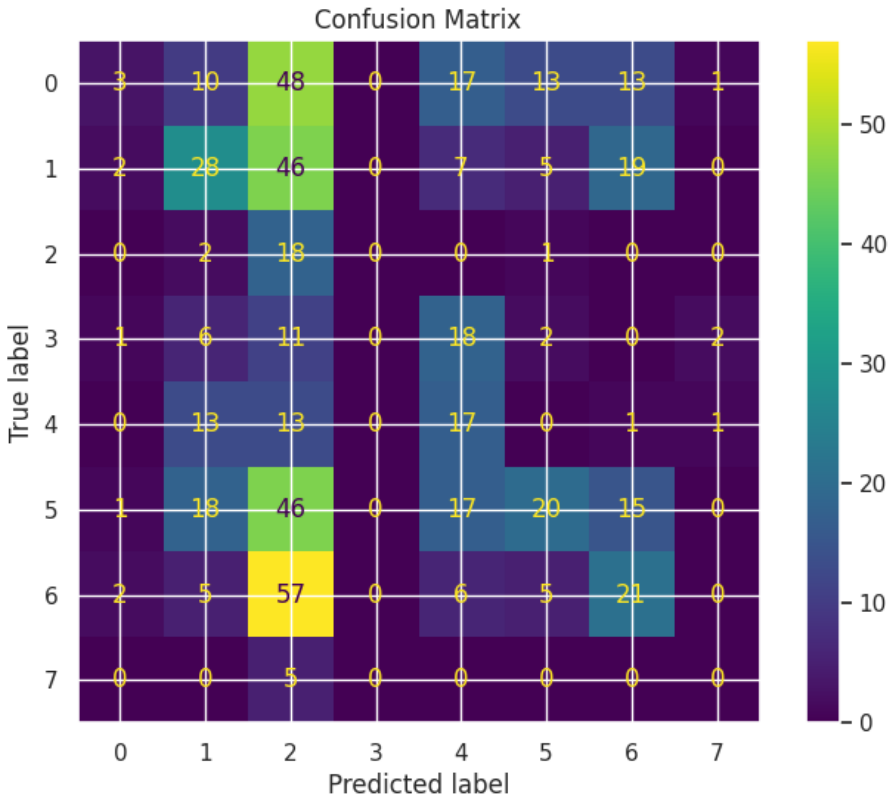


Figura 4. Matriz de Confusão da arquitetura GRU

3.3.3. Discussão

Com esses resultados e percepções em mente, partiu-se para uma validação local, considerando não apenas as métricas globais de desempenho, mas também a coerência dos resultados obtidos no contexto analisado, conforme a perspectiva na literatura [Kline 2016].

Inicialmente, ao observar a Tabela 1, nota-se que o TCN e suas variações híbridas não apresentaram bom desempenho nesse contexto, configurando-se como os modelos mais fracos. Isso ocorre pois seus valores de *gap* indicam *underfitting*, além de já apresentarem baixos valores de acurácia.

O FNN, por sua vez, apresentou maior estabilidade, sendo consistente nos resultados e com menor variação entre suas métricas. No entanto, essa estabilidade também indica que o modelo não é adequado para o contexto, uma vez que não consegue atingir valores mais elevados de desempenho.

Os modelos LSTM e GRU, individualmente, apresentaram os melhores resultados. Além disso, ao combinar LSTM e GRU em arquiteturas híbridas, observa-se uma

melhora no desempenho dos modelos (com exceção do FNN), tanto em acurácia quanto no gap. O LSTM, em especial, destacou-se como superior ao GRU nas combinações, empatando apenas no caso da CNN em termos de acurácia. Contudo, ao analisar o *gap*, percebe-se que a CNN apresenta sinais de *overfitting*, enquanto o GRU também sofre com esse problema. Assim, optou-se por eliminar combinações com GRU, visto que apresenta os mesmos sindais do LSTM, mas com valor de acurácia inferior.

Ao comparar com as versões não híbridas, como CNN e RNN, surge um impasse: apesar de apresentarem *gap* maior, foram selecionadas porque, nesse contexto, seus valores de acurácia são baixos, o que indica bom ajuste no treino (sem *underfitting*), já que isso implica maior acurácia no treino em relação à validação. Por outro lado, a média da validação cruzada indica valores relativamente altos, ainda que inconsistentes com a acurácia. Isso indica que, embora o modelo apresente bom desempenho médio durante a validação cruzada, esse comportamento não se mantém de forma consistente, mostrando instabilidade no modelo.

A métrica Kappa indica que todos os modelos apresentam desempenho fraco, reforçando que o problema está mais relacionado ao contexto e aos dados do que às arquiteturas em si. Nesse sentido, a avaliação dos modelos não foi baseada apenas nas métricas globais de desempenho, mas também na análise local dos resultados obtidos [Kline 2016].

Por fim, a métrica *F1-score* também indica baixo desempenho geral, embora LSTM e GRU se destaquem ligeiramente em relação aos demais.

Dessa forma, os modelos LSTM, GRU, CNN+LSTM e RNN+LSTM foram considerados os melhores nesse contexto. No entanto, mesmo esses modelos não atingiram valores elevados de desempenho, indicando que, em trabalhos futuros, seria necessário melhorar o banco de dados — seja tornando-o mais robusto, seja adotando outras estratégias —, visto que o uso de aumento de dados não foi suficiente, e que esses modelos apresentam comportamentos semelhantes, sugerindo que melhorias em um tenderiam a refletir nos demais.

Seguindo com a análise, pode-se remover o LSTM, visto que o *gap* está negativo, indicando que o treino foi ruim, visto que sua acurácia no teste é maior. Dessa forma, pode-se priorizar o CNN+LSTM em relação ao RNN+LSTM, visto que ele apresenta desempenho ligeiramente melhor. No fim, seria interessante utilizar uma versão híbrida e uma não híbrida para testes em trabalhos futuros nesse contexto.

Assim, o ideal seria testar inicialmente um desses modelos com o problema devidamente resolvido. Caso os resultados sejam positivos, os modelos poderiam ser comparados para identificar o superior, enquanto LSTM, CNN, RNN e RNN+LSTM ainda poderiam ser considerados como alternativas secundárias.

Todavia, considerando o baixo desempenho geral dos modelos e seguindo a literatura [Chawla et al. 2002, Breiman 2001, Krawczyk 2016], recomenda-se como trabalhos futuros: (i) a aplicação de técnicas de balanceamento de classes; (ii) a experimentação com modelos clássicos de aprendizado de máquina — como *Random Forest*, por exemplo — que tendem a apresentar maior robustez frente a desbalanceamentos de classe; e (iii) a coleta de dados adicionais, especialmente para as classes que apresentaram baixo desempenho na matriz de confusão, visando melhorar sua representatividade e, consequentemente, a eficácia geral da classificação.

4. Conclusões

Este estudo conduziu com sucesso uma análise comparativa entre redes neurais na base de dados do *Overlord*, visando permitir futuramente a aplicação desse arcabouço como um sistema de adaptação dinâmica de conteúdo. Foi possível identificar quais métricas e combinações obtiveram melhores resultados, permitindo que etapas desnecessárias sejam evitadas em trabalhos futuros. Além disso, algumas redes foram consideradas menos promissoras para esse contexto, conferindo vantagem para as arquiteturas GRU e CNN-LSTM, ou seja, apresentaram resultados relativamente mais promissores dentro das limitações do conjunto de dados utilizado.

No entanto, evidenciou-se que o banco de dados atual é insuficiente para alcançar bons resultados. Portanto, trabalhos posteriores devem investigar formas de suprir essas lacunas, incluindo: melhoria da qualidade do conjunto de dados, aplicação de técnicas de balanceamento, uso de aprendizado de máquina clássico ou coleta adicional de dados. Recomenda-se também testar o modelo diretamente na aplicação para avaliar a reação dos usuários, constituindo um método de validação mais realista. Uma vez corrigidas essas questões, outras variações de parâmetros poderão ser testadas para aprimorar o modelo — como o uso da função tanh em substituição à ReLU — além de novas combinações ou extensões de modelo para explorar os limites de seu desempenho. Consequentemente, uma vez definido e validado o modelo, a rede neural conseguirá atingir seu objetivo de identificar padrões de perfis neste contexto e, assim, auxiliar na compreensão das preferências dos jogadores de cada perfil.

Referências

An, M., Wu, F., Wu, C., Zhang, K., Liu, Z., and Xie, X. (2019). Neural news recommendation with long- and short-term user representations.

Azevedo, V., Nedjah, N., and Mourelle, L. (2020). Identificação do perfil de clientes utilizando redes neurais convolucionais. pages 1–8.

Balda, H. D. F. (2020). Improvement of player experience through dynamic difficulty adjustment using behavioral patterns and machine learning. Diss. (graduação), Tokyo University of Technology.

Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *J. Mach. Learn. Res.*, 13(null):281–305.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Breiman, L. (2001). Random forests. *Machine Learning*, 45:5–32.

Chanel, G. and Lopes, P. (2020). User evaluation of affective dynamic difficulty adjustment based on physiological deep learning. In *Augmented Cognition. Theoretical and Technological Approaches*, pages 3–23, Cham. Springer International Publishing.

Chawla, N., Bowyer, K., Hall, L., and Kegelmeyer, W. (2002). Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Chrysafiadi, K., Kamitsios, M., and Virvou, M. (2023). Fuzzy-based dynamic difficulty adjustment of an educational 3d-game. *Multimedia Tools and Applications*, 82(18):27525–27549.

de Lima, E. S., Silva, B. M., and Galam, G. T. (2022). Adaptive virtual reality horror games based on machine learning and player modeling. *Entertainment Computing*, 43:100515.

Global Growth Insights (2024). Video game market size, share, growth, and industry analysis, by types (kids, adults), applications (action, adventure and role playing, arcade, strategy, simulation, driving, other), and regional insights and forecast to 2035. `https://www.globalgrowthinsights.com/pt/market-reports/video-game-market-107798`. Acesso em: 16 mar. 2026.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. `http://www.deeplearningbook.org`.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, 2 edition.

Heinz, S., Bracher, C., and Vollgraf, R. (2017). An lstm-based dynamic customer model for fashion recommendation.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9:1735–1780.

Kline, R. B. (2016). *Principles and Practice of Structural Equation Modeling*. The Guilford Press, New York, 4 edition.

Kodiyan, D., Hardegger, F., Neuhaus, S., and Cieliebak, M. (2017). Author profiling with bidirectional rnns using attention with grus. In Cappellato, L., Ferro, N., Goeuriot, L., and Mandl, T., editors, *Working Notes of CLEF 2017 - Conference and Labs of the Evaluation Forum, Dublin, Ireland, September 11-14, 2017*, CEUR Workshop Proceedings. CEUR-WS.org.

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. volume 14.

Krawczyk, B. (2016). Learning from imbalanced data: Open challenges and future directions. *Progress in Artificial Intelligence*, 5.

Liapis, A., Yannakakis, G. N., Nelson, M. J., Preuss, M., and Bidarra, R. (2019). Orchestrating game generation. *IEEE Transactions on Games*, 11(1):48–68.

Martins, B. d. H. (2021). Assessing game player profiles through agent-based simulation.

Miyazaki, C. K. (2017). Redes neurais convolucionais para aprendizagem e reconhecimento de objetos 3d. Acesso em: 18/dec./2025.

Pereira, L., Viana, B., and Toledo, C. (2022). A system for orchestrating multiple procedurally generated content for different player profiles. *IEEE Transactions on Games*, PP:1–11.

Pereira, L. T., Teoi, T. Y., and Toledo, C. F. M. (2025). Evaluating a procedural content orchestrator gameplay data and classifying user profiles. *Journal on Interactive Systems*, 16(1):490–504.

Pereira, L. T., Viana, B. M. F., and Toledo, C. F. M. (2024). A system for orchestrating multiple procedurally generated content for different player profiles. *IEEE Transactions on Games*, 16(1):64–74.

Prince, S. J. D. (2023). *Understanding Deep Learning*. MIT Press.

Ricci, F., Rokach, L., and Shapira, B. (2010). *Recommender Systems Handbook*, volume 1-35, pages 1–35.

Romero-Mendez, E., Santana-Mancilla, P., Garcia-Ruiz, M., Montesinos-López, O., and Anido-Rifón, L. (2023). The use of deep learning to improve player engagement in a video game through a dynamic difficulty adjustment based on skills classification. *Applied Sciences*, 13:8249.

Rosa, M. P., Castanho, C., Silva, T., Sarmet, M., and Jacobi, R. (2023). *Dynamic Difficulty Adjustment by Performance and Player Profile in Platform Game*, pages 3–16.

Rymarczyk, P., Golabek, P., Sylwia, and Rzemieniak, M. (2021). Profiling and segmenting clients with the use of machine learning algorithms. *EUROPEAN RESEARCH STUDIES JOURNAL*, XXIV:513–522.

Schrader, C. (2022). *Serious Games and Game-Based Learning*, pages 1–14. Springer Singapore, Singapore.

Shaker, N., Togelius, J., and Nelson, M. J. (2016). Procedural content generation in games.

Teoi, T., Pereira, L., and Toledo, C. (2024). Towards adapting a content orchestrator to a different gamegenre: Generating levels, rules, and narrative for diverse player profiles from a top-down adventure to a 2d platformer. In *Anais do XXIII Simpósio Brasileiro de Jogos e Entretenimento Digital*, pages 396–415, Porto Alegre, RS, Brasil. SBC.

Yee, N. and Ducheneaut, N. (2018). Gamer motivation profiling: uses and applications. In Drachen, A., Mirza-Babaei, P., and Nacke, L., editors, *Games User Research*. Oxford University Press.

You, J., Wang, Y., Pal, A., Eksombatchai, P., Rosenburg, C., and Leskovec, J. (2019). Hierarchical temporal convolutional networks for dynamic recommender systems. In *The World Wide Web Conference*, WWW '19, page 2236–2246, New York, NY, USA. Association for Computing Machinery.

Zhou, X., Li, F., Hu, J., and Liang, J. (2025). A customer profiling method for personalized service in retail pharmacies. In *Proceedings of the 2025 6th International Conference on Computer Information and Big Data Applications*, CIBDA '25, page 1082–1086, New York, NY, USA. Association for Computing Machinery.